CRICKETIQ

Machine Learning Based Cricket Match Prediction and Analytics

CASE STUDY REPORT

Project Information	Details
Project Name	CricketIQ
Project Type	Machine Learning Web Application
Domain	Sports Analytics / Machine Learning
Model	Logistic Regression
Backend	Python + Flask
Deployment	Render Web Service
Student	VIGACINI C
Roll No.	23BIT126
Date	26 September 2026

1. Introduction

Cricket matches are influenced by several measurable and contextual factors, including historical team performance, recent form, head-to-head results, venue performance, and toss-related decisions. CricketIQ was developed as a machine-learning based web application that converts historical patterns into an interpretable estimate of the likely match winner.

The project combines a structured historical cricket dataset, feature engineering, Logistic Regression, and a Flask-based user interface. It provides prediction and analytics views so users can examine the statistics behind a prediction rather than receiving only a final class label.

2. Problem Statement

Cricket outcomes are difficult to estimate consistently because multiple performance factors interact. Although historical match information may be available, users need a structured mechanism to combine relevant variables and generate a probability-based prediction. CricketIQ addresses this by using historical and contextual cricket information to estimate the winning team and present supporting analytics.

3. Objectives

- Analyze historical cricket match information and identify useful predictive variables.
- Engineer comparable team-performance features using differences between Team 1 and Team 2.
- Train and evaluate a Logistic Regression classification model.
- Generate class probabilities and an estimated winning team.
- Provide historical, recent-form, head-to-head, and venue analytics.
- Deploy the application as a publicly accessible web service.

4. Dataset and Feature Engineering

The prepared historical dataset contains 2,259 match records and 23 variables, including teams, venue, toss information, winner, team-performance rates, and engineered difference features.

Category	Examples
Match information	Season, date, teams, venue
Toss information	Toss winner, toss decision
Outcome	Winner / target
Historical performance	Team historical win rates
Recent performance	Recent win rates
Head-to-head	Prior meeting win rates
Venue performance	Team performance at selected venue

4.1 Final Model Features

- Historical Win Rate Difference = Team 1 historical rate − Team 2 historical rate.
- Recent Win Rate Difference = Team 1 recent rate − Team 2 recent rate.
- Head-to-Head Win Rate Difference = Team 1 H2H rate − Team 2 H2H rate.
- Venue Win Rate Difference = Team 1 venue rate − Team 2 venue rate.
- Toss Decision = Bat First / Field First, encoded numerically.

Where prior history is unavailable, the feature-engineering logic uses a neutral fallback value of 0.5 for the corresponding rate.

4.2 Model Scope

The application collects the toss winner, but the identity of the toss winner is not an independent input to the saved Logistic Regression model. The current model uses the encoded toss decision instead.

5. Machine Learning Methodology

CricketIQ follows a supervised binary-classification workflow. Historical match records are transformed into numerical features, standardized, and supplied to a Logistic Regression classifier. The trained model produces probabilities for the two outcome classes.

Stage	Description

Historical data	Match records and team-performance information
Feature engineering	Historical, recent, H2H and venue differences
Scaling	StandardScaler transforms the five model inputs
Classification	Logistic Regression predicts the outcome class
Output	Predicted winner and class probabilities

5.1 Logistic Regression

Logistic Regression was selected because it supports binary classification and provides interpretable coefficients. The model estimates the probability of each outcome from the standardized feature vector.

5.2 Prediction Logic

- Collect Team 1, Team 2, venue, toss winner, and toss decision.
- Calculate the required historical/contextual features.
- Construct the five-feature input vector in the model's expected order.
- Scale the input and generate the predicted class and probabilities.
- Display the estimated winner, team probabilities, and supporting factors.

6. Model Performance and Analysis

The final Logistic Regression model was evaluated using standard classification metrics.

Metric	Result	Meaning
Accuracy	62.83%	Overall proportion of correct predictions.
Precision	61.92%	Precision of positive-class predictions.
Recall	77.92%	Ability to identify positive-class outcomes.
F1-Score	69.00%	Balance of precision and recall.
ROC-AUC	67.43%	Discrimination across thresholds.

6.1 Model Coefficients

Feature	Coefficient	Direction
Historical win-rate difference	+0.513	Positive
Recent win-rate difference	+0.184	Positive
H2H win-rate difference	+0.167	Positive
Venue win-rate difference	+0.118	Positive
Toss decision	−0.054	Negative in fitted encoding

Historical win-rate difference has the largest positive coefficient among the five model inputs. Coefficients should be interpreted in the context of standardized inputs and the fitted dataset, not as causal effects.

6.2 Reconstructed Confusion Matrix

The exact confusion matrix was not retained with the final saved model artifacts. A matrix reconstructed from the rounded reported metrics and a 452-record holdout is therefore not treated as the exact original matrix. The reconstructed counts are 97, 115, 53, and 187.

7. CricketIQ Web Application

The trained model is integrated into a Flask web application organized into prediction, analytics, and model-information views.

7.1 Match Prediction

- Searchable Team 1 and Team 2 selection.
- Searchable venue selection.
- Toss winner and Bat First / Field First selection.
- Prediction result with estimated team probabilities.

7.2 Analytics Module

- Historical win rate, recent form, head-to-head and venue performance.
- Recent five-match information, historical counts and H2H records.

7.3 Model Module

- Logistic Regression explanation, metrics, coefficients and confusion-matrix visualization.

7.4 Technology Stack

Technology	Purpose
Python	Model integration and backend logic
Flask	Web application and routing
Pandas	Data processing and feature lookup
Scikit-learn	Scaling and Logistic Regression
Joblib	Saving/loading trained artifacts
HTML / CSS / JavaScript	Frontend and interaction
Gunicorn	Production WSGI server
Render	Public web deployment

8. System Deployment

CricketIQ was deployed as a Flask Web Service on Render. The GitHub repository contains the application source, trained model, scaler, dataset, templates, static assets, and dependency specification.

Component	Configuration
Service type	Render Web Service
Source	GitHub: svegithub/CricketIQ
Branch	main
Build command	pip install -r requirements.txt
Start command	gunicorn app:app
Runtime	Python 3
Tier	Free tier prototype

8.1 Runtime Flow

- User accesses the public CricketIQ service.
- Flask receives the selected match parameters.
- The application derives the required historical/contextual features.
- The saved scaler transforms the five model features.
- The saved Logistic Regression model generates the prediction and probabilities.
- The result is rendered in the browser with supporting analytics.

9. Testing and Validation

Testing focused on the prediction workflow, input handling, feature direction, page navigation, model loading, and deployment behavior.

Test Area	Expected Result	Status
Model loading	Saved model and scaler load successfully.	Verified
Prediction input	Selected teams, venue and toss values are processed.	Verified
Team swap behavior	Directional feature differences update consistently.	Verified
Probability output	Team probabilities are generated from model probabilities.	Verified
Analytics	Historical, recent, H2H and venue information is displayed.	Verified
Navigation	Application pages open correctly.	Verified
Public deployment	Application is accessible through Render.	Verified

10. Key Findings

- Historical team strength is the strongest positive contributor among the current model features.
- Recent form, head-to-head performance, and venue performance add contextual information.
- Logistic Regression provides an interpretable approach through model coefficients.
- The application combines prediction with analytics, allowing users to inspect supporting statistics.
- The system demonstrates an end-to-end machine-learning deployment workflow.

11. Limitations

- The model is based on historical match information and does not represent every factor affecting a cricket match.
- Player-level statistics, playing XI information, injuries, weather, pitch conditions, and live match events are not included.
- The saved model does not use toss-winner identity as an independent feature, even though the application collects it.
- Reported metrics represent the evaluated dataset and should not be interpreted as guaranteed future performance.
- The exact original confusion matrix was not preserved; the matrix used in the report/application is explicitly reconstructed.
- The application is a prototype and does not use live cricket feeds.

12. Future Enhancements

- Retrain the model with player form, playing XI, pitch, weather, and toss-winner identity.
- Compare Logistic Regression with tree-based and ensemble classifiers.
- Introduce probability calibration and stronger validation procedures.
- Integrate a reliable live or regularly updated cricket data source.
- Add player-level and match-condition analytics.
- Add model monitoring and periodic retraining.
- Improve deployment scalability and operational monitoring.

13. Practical Relevance

CricketIQ demonstrates how a conventional machine-learning classifier can be converted into a usable sports-analytics product. The project connects data preparation, feature engineering, statistical learning, probability estimation, interface design, and cloud deployment in one workflow. The same pattern can be adapted to other sports prediction and decision-support applications where historical structured data is available.

14. Conclusion

CricketIQ demonstrates an end-to-end approach to cricket match prediction using historical data and Logistic Regression. The project begins with structured match information, derives comparable team-performance features, standardizes the model inputs, and produces probability-based predictions through a Flask web application.

The completed application extends beyond a prediction screen by providing analytics for historical strength, recent form, head-to-head performance, and venue performance. It was also successfully deployed as a public web service, making the project demonstrable as both a machine-learning case study and a software application.

The current system should be understood as a predictive prototype rather than a definitive match-outcome engine. Future development can focus on richer cricket features, live data, model comparison, probability calibration, and improved deployment capabilities.

15. References

- Scikit-learn – Logistic Regression documentation: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- Scikit-learn – Metrics documentation: https://scikit-learn.org/stable/api/sklearn.metrics.html
- Flask – Quickstart documentation: https://flask.palletsprojects.com/en/stable/quickstart/
- Render – Deploy a Flask App: https://render.com/docs/deploy-flask